



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 26 de marzo del 2026

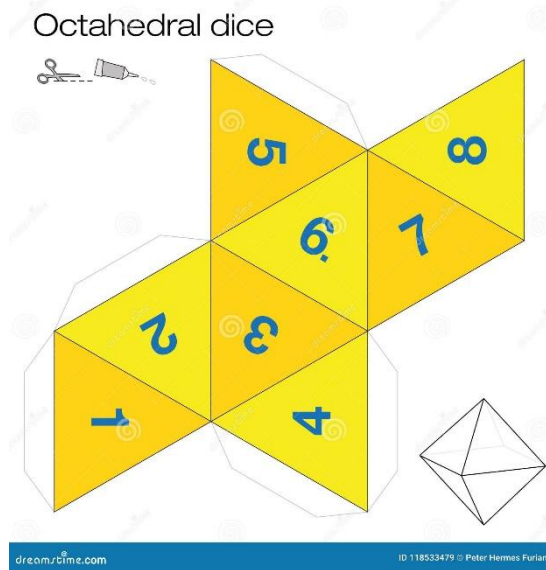
CALIFICACIÓN: _____

ACTIVIDADES

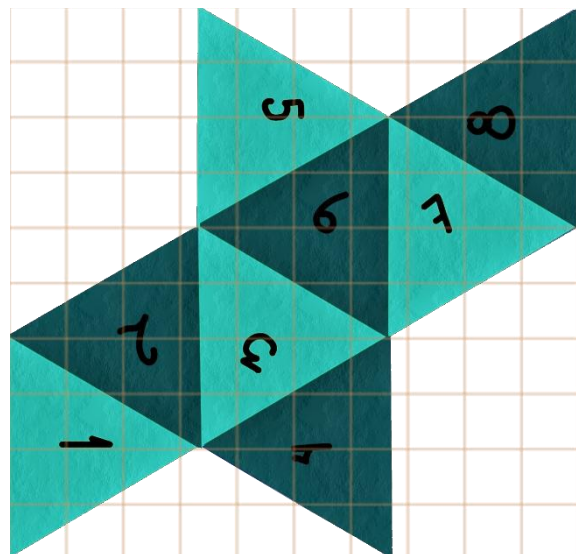
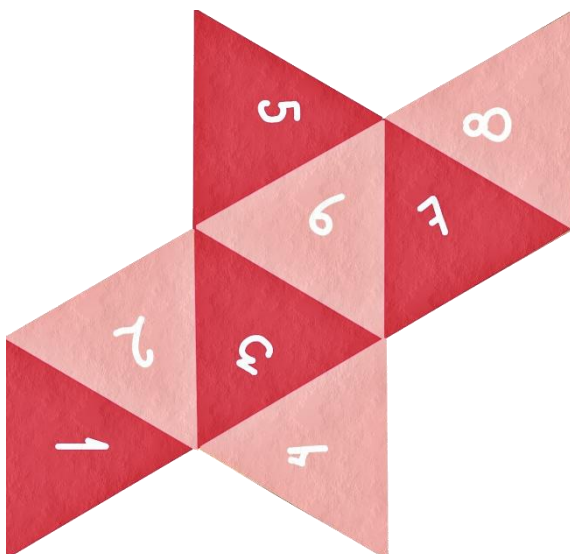
Para esta práctica numero 6 se solicitaron tres actividades referentes al texturizado que se presentan a continuación

OCTAEDRO

Como primera actividad se pidió realizar un dado de 8 caras (mejor conocido como octaedro) y texturizarlo mediante código. Para ello primero se busco una imagen de referencia que se pudiera utilizar para “envolver” nuestra figura, obteniendo por resultado la siguiente imagen de internet.



Sin embargo, se decidió modificar la imagen mediante uso de programas externos como GIMP y Photoscape para activar el canal alfa, escalar la imagen a una potencia de 2 y eliminar contornos y marcas de agua, obteniendo por resultado la imagen de la izquierda, mientras que la imagen de la derecha presentada se usaría como base para saber las coordenadas donde texturizar cada cara.



Dentro del código se realizaron las siguientes modificaciones.

Window.h

Para observar bien el octaedro sin necesidad de mover de manera tan específica la cámara se hicieron los cambios a los cuales ya estamos acostumbrados, se activaron rotaciones completas en nuestro dado.

```
Public:
    GLfloat getRotacionDado() { return rotacionDado; } // AGREGAMOS LA ROTACIÓN DEL DADO
Private:
    GLfloat rotacionDado; // AGREGAMOS LA ROTACIÓN DEL DADO
```

Window.cpp

Instanciamos nuestras rotaciones y las teclas con las que las manejamos, siendo para el dado la tecla F y V sin tope alguno.

```
Window::Window()
    rotacionDado = 0.0f;
Window::Window(GLint windowHeight, GLint windowHeight)
    rotacionDado = 0.0f;
void Window::ManejaTeclado(...)
    if (key == GLFW_KEY_F)
    {
        theWindow->rotacionDado -= 5.0f; // Rota hacia un lado (5 grados)
    }
    if (key == GLFW_KEY_V)
    {
        theWindow->rotacionDado += 5.0f; // Rota hacia el lado contrario
    }
```

Main

Ya que traeremos una textura externa, al inicio de nuestro código instanciamos una nueva

```
Texture OctaedroTexture;
```

Hacemos una función para crear nuestro octaedro donde pasamos los vértices X,Y,Z las variables S y T (Para marcar las coordenadas x,y de nuestra textura/imagen a usar) y las variables NX, NY, NZ correspondientes a la luz que de momento no usaremos.

```
GLfloat octa_vertices[] = {
// x      y      z      S      T      NX      NY      NZ
// ===== MITAD SUPERIOR (Triángulos apuntando hacia arriba) =====
// Cara 1: Superior
0.0f, 0.5f, 0.0f, 0.32f, 0.20f, 0.5f, 0.5f, 0.5f, // Vértice Superior
0.0f, 0.0f, 0.5f, 0.01f, 0.39f, 0.5f, 0.5f, 0.5f, // Frente
0.5f, 0.0f, 0.0f, 0.01f, 0.01f, 0.5f, 0.5f, 0.5f, // Derecha

// Cara 2: Superior
0.0f, 0.5f, 0.0f, 0.33f, 0.21f, 0.5f, 0.5f, -0.5f, // Vértice Superior
0.5f, 0.0f, 0.0f, 0.33f, 0.59f, 0.5f, 0.5f, -0.5f, // Derecha
0.0f, 0.0f, -0.5f, 0.01f, 0.41f, 0.5f, 0.5f, -0.5f, // Atrás

// Cara 3: Superior
0.0f, 0.5f, 0.0f, 0.34f, 0.21f, -0.5f, 0.5f, -0.5f, // Vértice Superior
0.0f, 0.0f, -0.5f, 0.65f, 0.40f, -0.5f, 0.5f, -0.5f, // Atrás
-0.5f, 0.0f, 0.0f, 0.34f, 0.59f, -0.5f, 0.5f, -0.5f, // Izquierda

// Cara 4: Superior
```

```

0.0f,    0.5f,    0.0f,           0.35f,    0.21f,    -0.5f,    0.5f,    0.5f, // Vértice Superior
-0.5f,   0.0f,   0.0f,           0.67f,    0.01f,    -0.5f,    0.5f,    0.5f, // Izquierda
0.0f,    0.0f,   0.5f,           0.67f,    0.40f,    -0.5f,    0.5f,    0.5f, // Frente

// ===== MITAD INFERIOR (Triángulos apuntando hacia abajo) =====

// Cara 8: Inferior
0.0f,   -0.5f,   0.0f,           0.67f,    0.80f,    -0.5f,   -0.5f,   -0.5f, // Vértice Inferior
-0.5f,   0.0f,   0.0f,           0.99f,    0.61f,    -0.5f,   -0.5f,   -0.5f, // Izquierda
0.0f,    0.0f,  -0.5f,           0.99f,    0.99f,    -0.5f,   -0.5f,   -0.5f, // Atrás

// Cara 7: Inferior
0.0f,   -0.5f,   0.0f,           0.67f,    0.79f,    0.5f,   -0.5f,    0.5f, // Vértice Inferior
0.0f,    0.0f,  -0.5f,           0.67f,    0.41f,    0.5f,   -0.5f,    0.5f, // Derecha
0.5f,    0.0f,   0.0f,           0.98f,    0.60f,    0.5f,   -0.5f,    0.5f, // Frente

// Cara 5: Inferior
0.0f,   -0.5f,   0.0f,           0.66f,    0.80f,    0.5f,   -0.5f,   -0.5f, // Vértice Inferior
0.0f,    0.0f,   0.5f,           0.33f,    0.99f,    0.5f,   -0.5f,   -0.5f, // Atrás
-0.5f,   0.0f,   0.0f,           0.33f,    0.61f,    0.5f,   -0.5f,   -0.5f, // Derecha

// Cara 6: Inferior
0.0f,   -0.5f,   0.0f,           0.66f,    0.79f,    -0.5f,   -0.5f,    0.5f, // Vértice Inferior
0.5f,    0.0f,   0.0f,           0.34f,    0.61f,    -0.5f,   -0.5f,    0.5f, // Frente
0.0f,    0.0f,   0.5f,           0.66f,    0.41f,    -0.5f,   -0.5f,    0.5f, // Izquierda

};

Mesh* octaedro = new Mesh();
// Son 24 vértices en total, por 8 datos cada uno (x,y,z, s,t, nx,ny,nz) = 192.
// Y 24 índices que conforman los 8 triángulos.
octaedro->CreateMesh(octa_vertices, octa_indices, 192, 24);
meshList.push_back(octaedro); // Este será el índice [5] en tu meshList
}
// =====

```

Ya dentro de nuestra función main mandamos a llamar a nuestra función anterior y cargamos la imagen con extensión .tga para que funja como textura

```

CrearOctaedro();
OctaedroTexture = Texture("Textures/OCTAEDRO.tga");
OctaedroTexture.LoadTextureA();

```

Finalmente, dentro de nuestro bucle while por cada ciclo de reloj renderizamos nuestro dado, trasladándolo para que no choque con el coche de las siguientes actividades y activando la rotación en Y con las teclas F y V

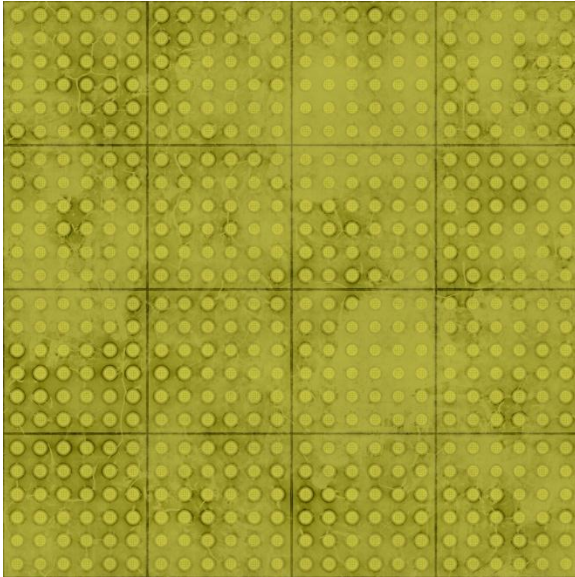
```

// =====
// Renderizado del Octaedro
model = glm::mat4(1.0);
// Lo trasladamos un poco a la derecha para que no choque con tus cubos
model = glm::translate(model, glm::vec3(0.0f, 5.0f, 8.0f));
model = glm::rotate(model, mainWindow.getRotacionDado() * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); // Para
que rote igual que tus dados
model = glm::scale(model, glm::vec3(4.0f, 4.0f, 4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
OctaedroTexture.UseTexture();
meshList[5]->RenderMesh();
// =====

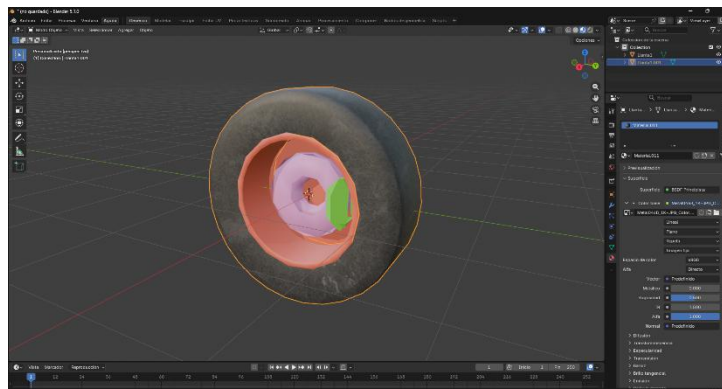
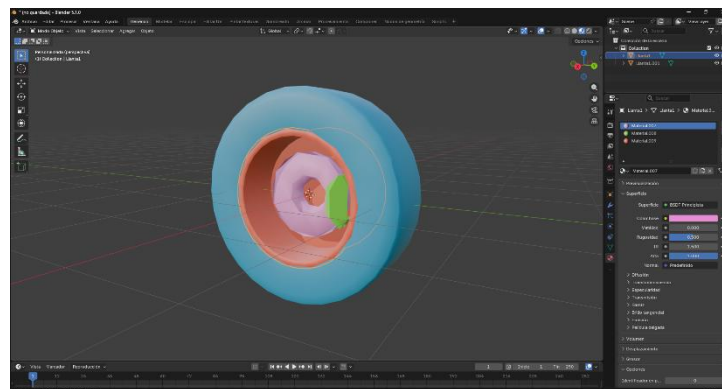
```

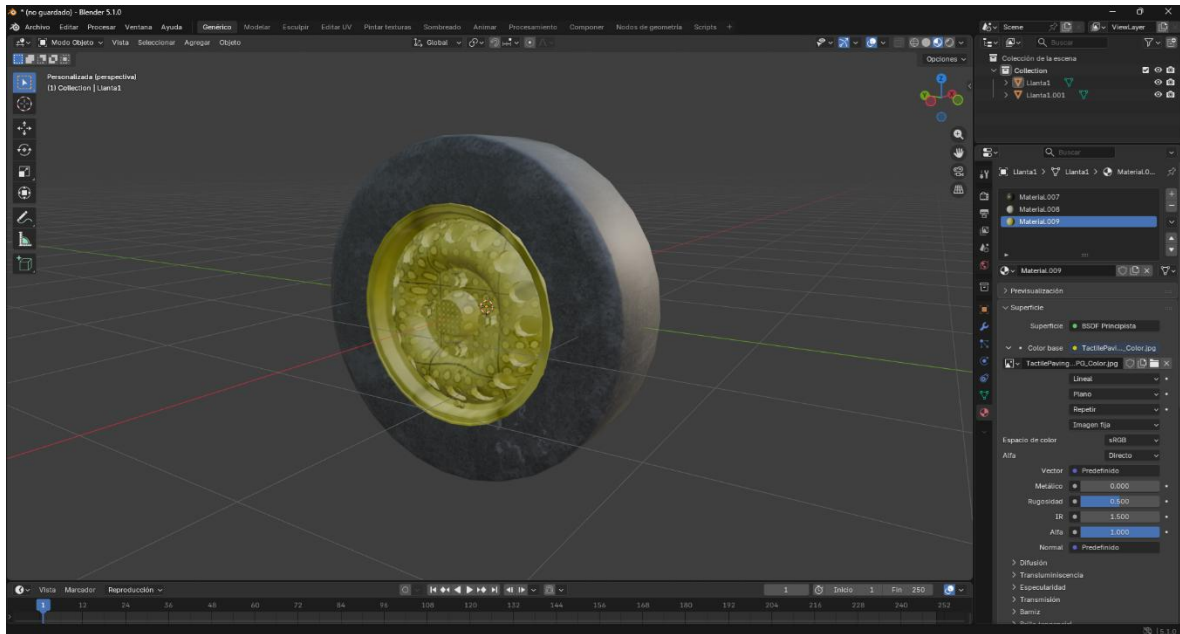
LLANTAS

Reutilizamos el modelo .obj que teníamos de la práctica anterior de nuestra llanta de coche, este texturizado fue sencillo mediante el uso de blender. Se utilizaron texturas de la página [ambientcg](#) mostrados a continuación.



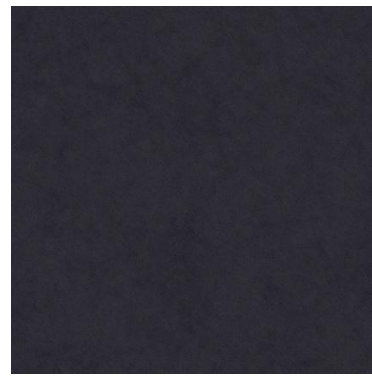
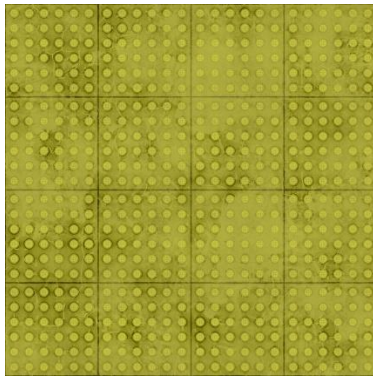
En blender, se seleccionaron distintas partes de la llanta y en la sección de materiales se cargaron las imágenes anteriormente mostradas.





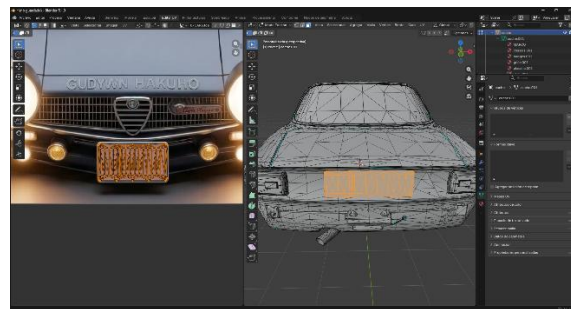
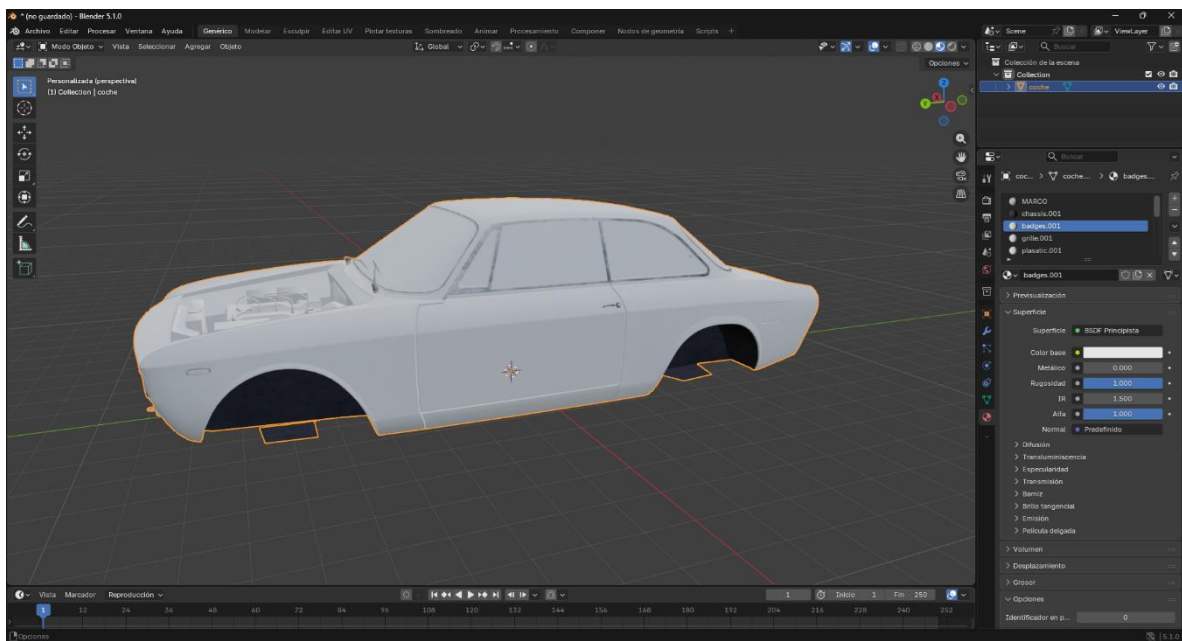
CARROCERÍA

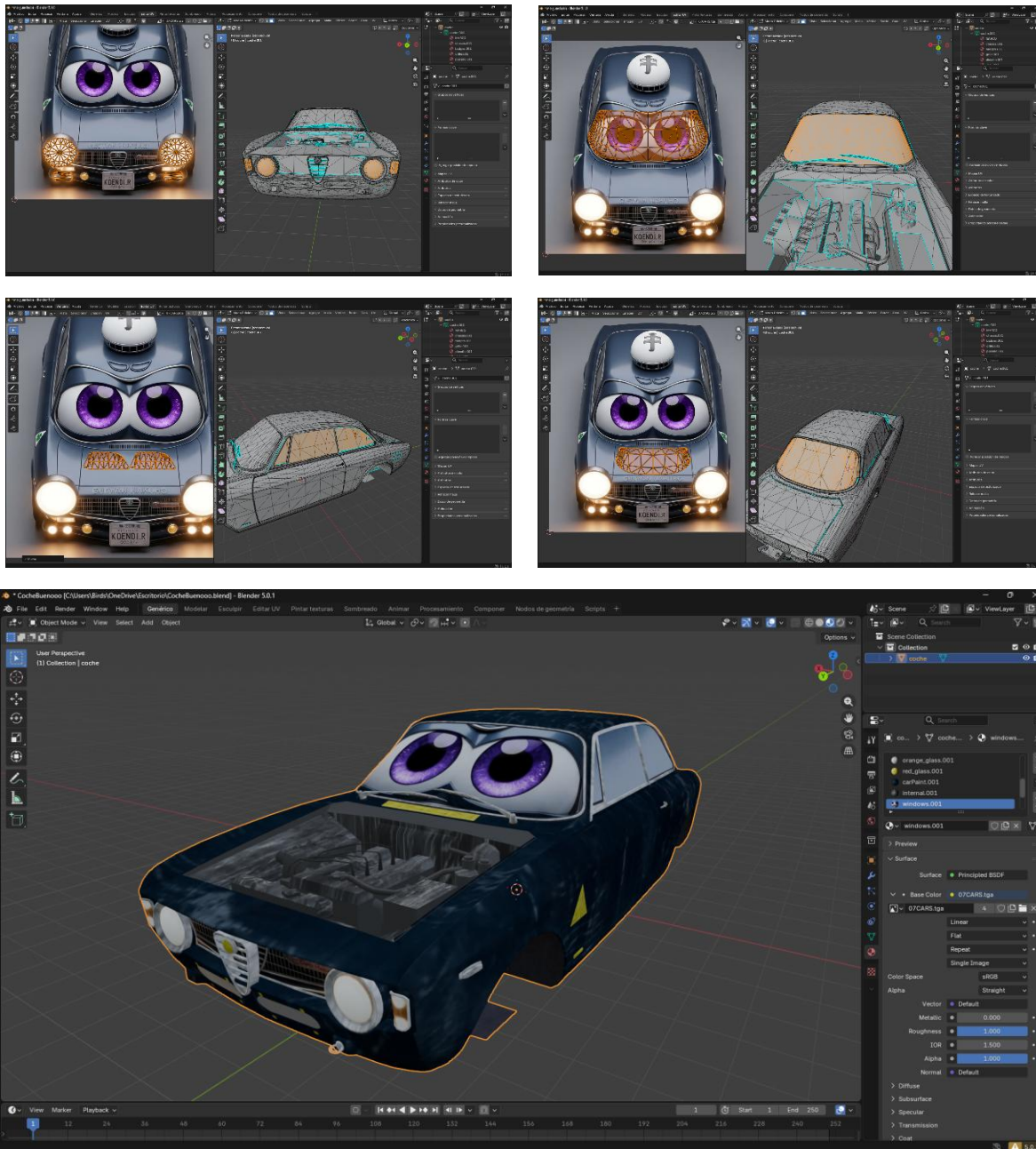
Para la carrocería no fue un proceso tan rápido como el anterior, también se utilizaron texturas de [ambientcg](#) sin embargo también se utilizó la imagen de coche cars entregada en el previo anterior.





A diferencia del cargado de texturas anterior, en este se tuvo que utilizar la función de mapas UV para ajustar los vértices a la imagen.





Una vez con ambos modelos texturizados procedemos a cargarlos en código haciendo algunas modificaciones.

Window

Manejaremos rotación en las llantas y traslación total del coche, además de la apertura del cofre, estas cosas fueron implementadas en la práctica anterior así que solo se vuelven a instanciar de la misma forma en Window.h y Window.cpp


```

Public:
    GLfloat getarticulacion1() { return articulacion1; } // Rotación Llantas
    GLfloat getarticulacion2() { return articulacion2; } // Apertura Cofre
    GLfloat getmovCoche() { return movCoche; } // Traslación Coche en Z
Private:
    // --- VARIABLES DE ANIMACIÓN DEL COCHE ---
    GLfloat articulacion1;
    GLfloat articulacion2;
    GLfloat movCoche;

Window::Window()
    articulacion1 = 0.0f;
    articulacion2 = 0.0f;
    movCoche = 0.0f;
Window::Window(GLint windowHeight, GLint windowHeight)
    articulacion1 = 0.0f;
    articulacion2 = 0.0f;
    movCoche = 0.0f;

void Window::ManejaTeclado(...)
    // Llantas y Coche avanza - Teclas H (Adelante) y N (Atrás)
    if (key == GLFW_KEY_H) {
        theWindow->articulacion1 += 5.0f;
        theWindow->movCoche -= 0.5f;
    }
    if (key == GLFW_KEY_N) {
        theWindow->articulacion1 -= 5.0f;
        theWindow->movCoche += 0.5f;
    }
    // Cofre - Teclas G (Abrir) y B (Cerrar)
    if (key == GLFW_KEY_G) {
        theWindow->articulacion2 += 2.0f;
        if (theWindow->articulacion2 > 60.0f) theWindow->articulacion2 = 60.0f;
    }
    if (key == GLFW_KEY_B) {
        theWindow->articulacion2 -= 2.0f;
        if (theWindow->articulacion2 < 0.0f) theWindow->articulacion2 = 0.0f;
    }
}

```

Main

Al traer modelos “nuevos” en nuestras primeras líneas de código instanciamos los modelos a usar, siendo estos la llanta texturizada, el cofre texturizado y el coche texturizado.

```

Model LlantaT;
Model CofreT;
Model CocheT;

```

Ya dentro de nuestra función main cargamos nuestros tres modelos .obj

```

LlantaT = Model();
LlantaT.LoadModel("Models/llantaT1.obj");
CocheT = Model();
CocheT.LoadModel("Models/CocheT.obj");
CofreT = Model();
CofreT.LoadModel("Models/CofreT.obj");

```

Finalmente, dentro de nuestro bucle while por cada ciclo de reloj renderizamos nuestro coche de manera jerárquica para aplicar rotaciones y traslaciones por igual, lo colocamos al lado del octaedro e instanciamos las 4 llantas a partir de un solo modelo y acomodamos el cofre.

```
// =====
//Instancia del coche
model = glm::mat4(1.0);
// Agregamos el movimiento del coche en Z al padre
model = glm::translate(model, glm::vec3(0.0f + mainWindow.getmovCoche(), -1.1f, 0.0f ));
modelaux = model; // Pivote central guardado
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Cochet.RenderModel();

//Llanta trasera derecha
model = modelaux;
model = glm::translate(model, glm::vec3(-4.3f, 0.4f, 3.1f));
// Rotación de la llanta (articulacion1) ANTES de acomodarla para que gire en su eje
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
color = glm::vec3(0.5f, 0.5f, 0.5f); //llanta con color gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
LlantaT.RenderModel();

//Llanta trasera izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(-4.3f, 0.4f, -3.1f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
LlantaT.RenderModel();

//Llanta delantera derecha
model = modelaux;
model = glm::translate(model, glm::vec3(6.0f, 0.4f, 3.1f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
LlantaT.RenderModel();

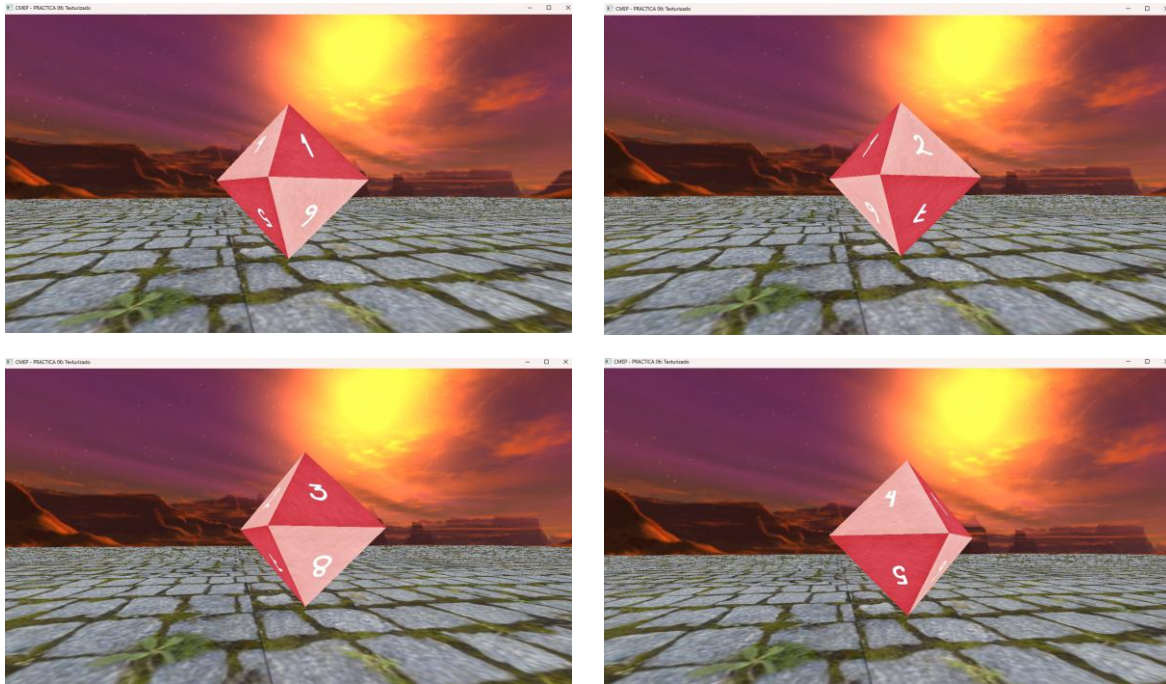
//Llanta delantera izquierda
model = modelaux;
model = glm::translate(model, glm::vec3(6.0f, 0.4f, -3.1f));
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::rotate(model, 90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
LlantaT.RenderModel();

//Cofre del coche
model = modelaux;
model = glm::translate(model, glm::vec3(4.2f, 2.6f, 0.0f));
model = glm::rotate(model, -90 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
// Rotación del cofre para abrir/cerrar (articulacion2)
model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
color = glm::vec3(0.5f, 0.5f, 0.5f); //llanta con color gris
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
CofreT.RenderModel();
// =====
```

EJECUCIÓN

Consideremos las teclas de movimiento para el octaedro como F y V; las teclas para el cofre del coche como G y B y las teclas para mover el coche como H y N.

Octaedro



Coche

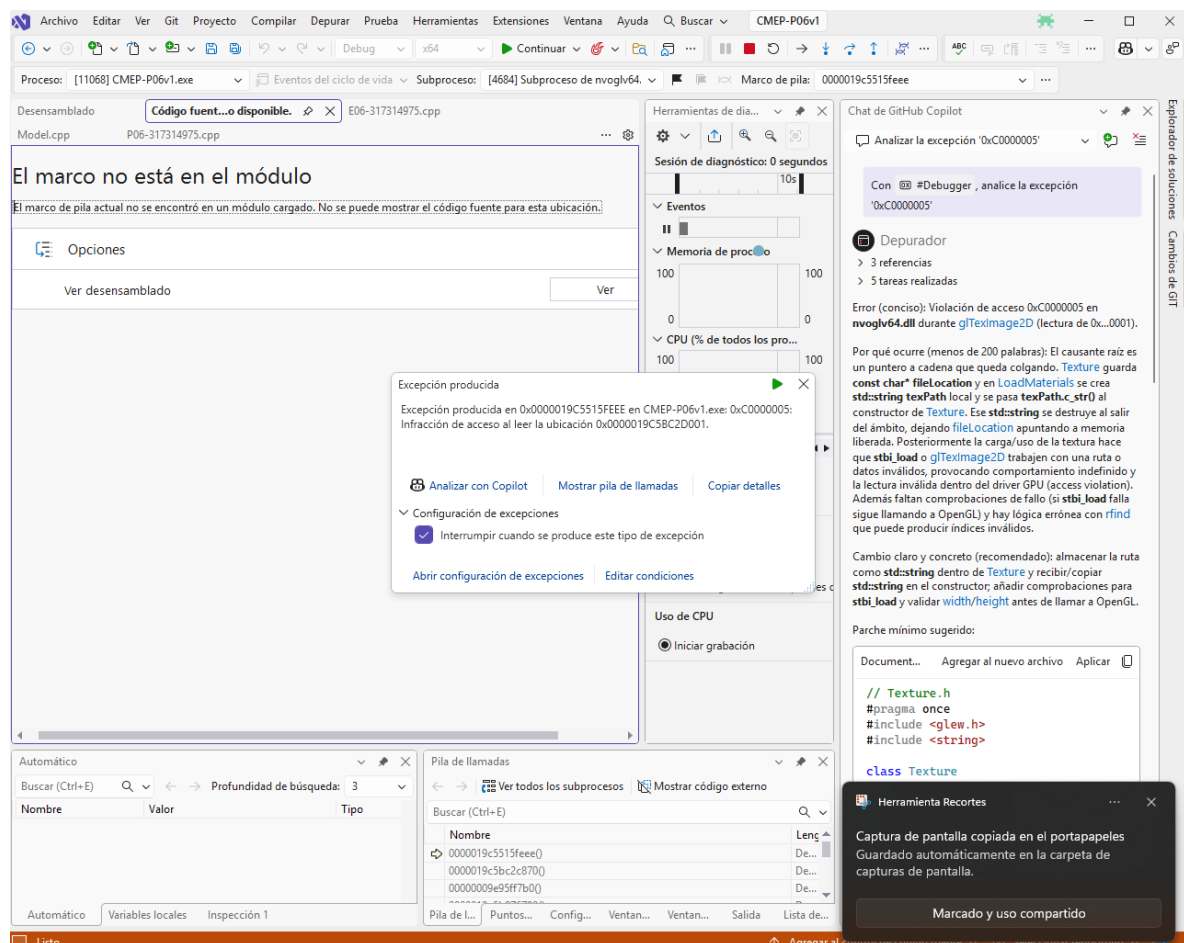


Para observar una mejor ejecución se adjunta el link de un GIF en drive.

<https://drive.google.com/file/d/17QQFkRv9rbPaTszEtVwnbrj9b-aPmEgX/view?usp=sharing>

PROBLEMAS PRESENTADOS

Excepción en tiempo de ejecución al cargar texturas en modelos importados: Durante la etapa de carga de los modelos 3D texturizados (exportados desde Blender) hacia OpenGL en visual, se presentó una excepción de memoria en tiempo de ejecución que provocaba el cierre abrupto del programa (una pantalla en negro). Al realizar la depuración, se identificó que el fallo ocurría dentro de la función de carga de la clase Texture (utilizando la librería stb_image), la cual no lograba procesar correctamente los datos de las imágenes originales en formatos comprimidos estándar, generando un conflicto de lectura.



SOLUCIÓN ENCONTRADA

Pudimos encontrar que el problema venía desde las imágenes que se usaban para texturizar, ya que faltaba activar el canal alfa. Para solucionarlo, se realizó un preprocesamiento de todas las imágenes de textura utilizando GIMP. El procedimiento consistió en abrir cada imagen, añadir y activar explícitamente el canal alfa (transparencia), y exportarlas en

formato .tga (Truevision TGA), el cual almacena los píxeles de forma no comprimida y es altamente compatible con las lecturas a nivel de bytes requeridas por OpenGL.

Posteriormente, estas nuevas imágenes .tga se reasignaron a los materiales correspondientes dentro del entorno de Blender. Al volver a exportar los archivos .obj y .mtl con las nuevas rutas hacia los archivos .tga, la librería de Assimp y el código en C++ lograron leer y mapear las texturas correctamente sin arrojar ninguna excepción.

CONCLUSIONES

Sobre los ejercicios

El trabajo con los ejercicios fue algo desafiante, especialmente cuando pasamos de texturizar de manera manual (como se hizo con las coordenadas del octaedro) a aplicar texturas en modelos más complejos que venían de otros programas. Asignar diferentes materiales a un solo objeto, como separar las partes de caucho y metal en las llantas, resultó ser un proceso detallado. Además, se vio que tener bien sincronizados los mapas UV y configurar adecuadamente las rutas de los archivos de materiales fue clave para que las texturas funcionaran correctamente con la librería Assimp en OpenGL.

Comentarios generales

En términos generales, la práctica cumplió su objetivo de aprendizaje, aunque hubiese sido útil recibir más explicaciones sobre cómo manejar los archivos y formatos. Sería recomendable poner más énfasis en algunos detalles técnicos, como la necesidad de que las imágenes tengan canales alfa en formatos como .tga y .png, y en cómo exportar los modelos correctamente. También, aprender más sobre cómo depurar y encontrar errores de lectura en las rutas de los archivos ayudaría a reducir el tiempo que se pierde resolviendo problemas relacionados con la memoria durante la compilación.

Cierre

En resumen, esta práctica fue un reto que, al final, resultó ser muy valiosa desde el punto de vista técnico. No solo se logró entender y aplicar el mapeo UV y las texturas con múltiples materiales para hacer que los modelos fueran más realistas, sino que también se logró integrar conceptos previos como las jerarquías y transformaciones matriciales. El resultado final, con la animación y el texturizado del vehículo, fue un paso importante que nos prepara para futuros proyectos más complejos en computación gráfica.

REFERENCIAS

- De Vries, J. (s.f.). *Textures*. LearnOpenGL. Recuperado el 23 de marzo de 2026, de <https://learnopengl.com/Getting-started/Textures>

- ambientCG. (s.f.). *Public Domain Resources for Physically Based Rendering*. Recuperado el 23 de marzo de 2026, de <https://ambientcg.com/>
- ambientCG. (s.f.). *TactilePaving001*. Recuperado el 23 de marzo de 2026, de <https://ambientcg.com/view?id=TactilePaving001>
- ambientCG. (s.f.). *Metal046B*. Recuperado el 23 de marzo de 2026, de <https://ambientcg.com/view?id=Metal046B>
- ambientCG. (s.f.). *Plastic001*. Recuperado el 23 de marzo de 2026, de <https://ambientcg.com/view?id=Plastic001>
- ambientCG. (s.f.). *Plastic005*. Recuperado el 23 de marzo de 2026, de <https://ambientcg.com/view?id=Plastic005>
- ambientCG. (s.f.). *Metal027*. Recuperado el 23 de marzo de 2026, de <https://ambientcg.com/view?id=Metal027>
- ambientCG. (s.f.). *HDRIElement004*. Recuperado el 23 de marzo de 2026, de <https://ambientcg.com/view?id=HDRIElement004>